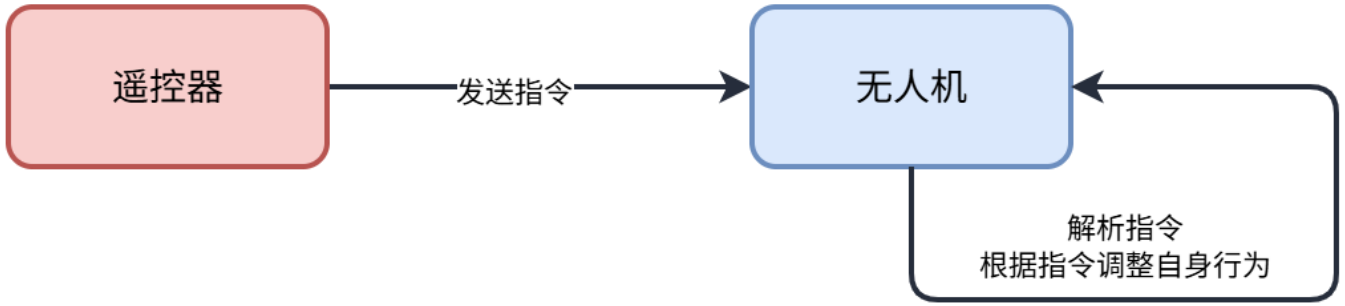
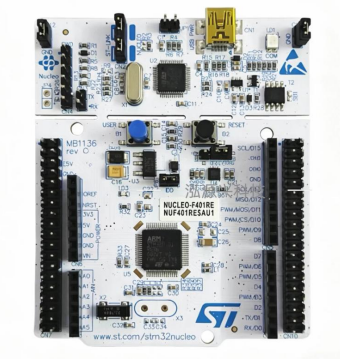
挑战点8：硬件子系统接收并解析遥控器命令



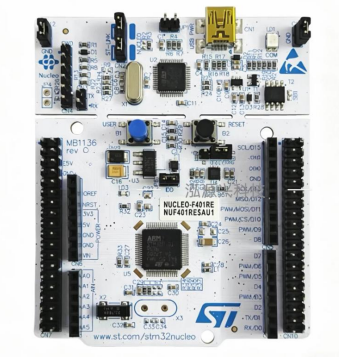


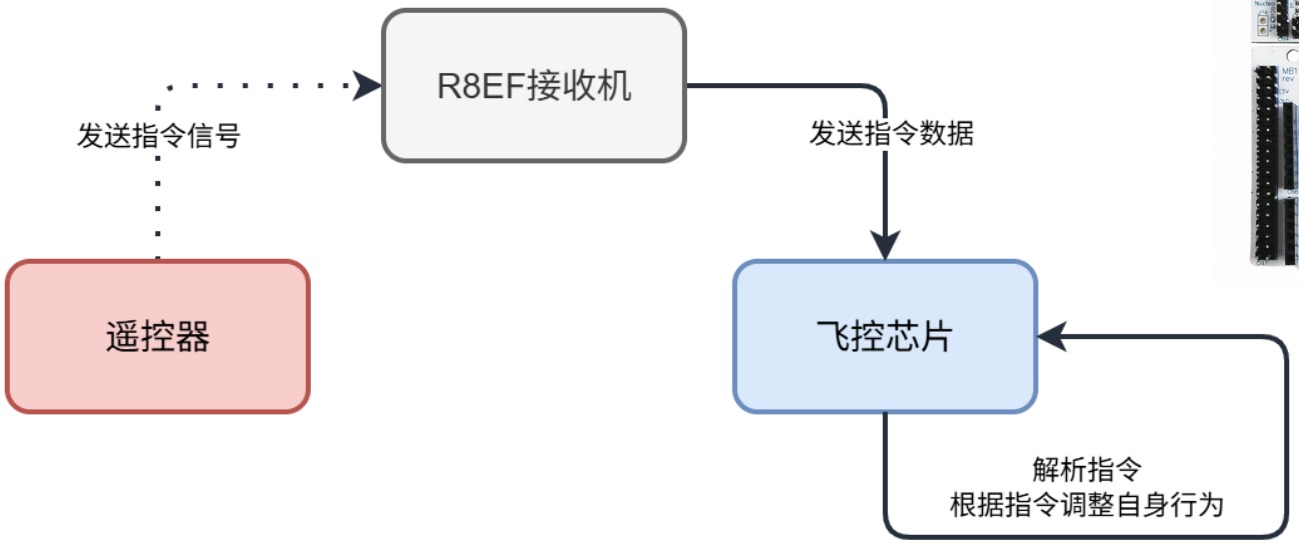


挑战点8：硬件子系统接收并解析遥控器命令









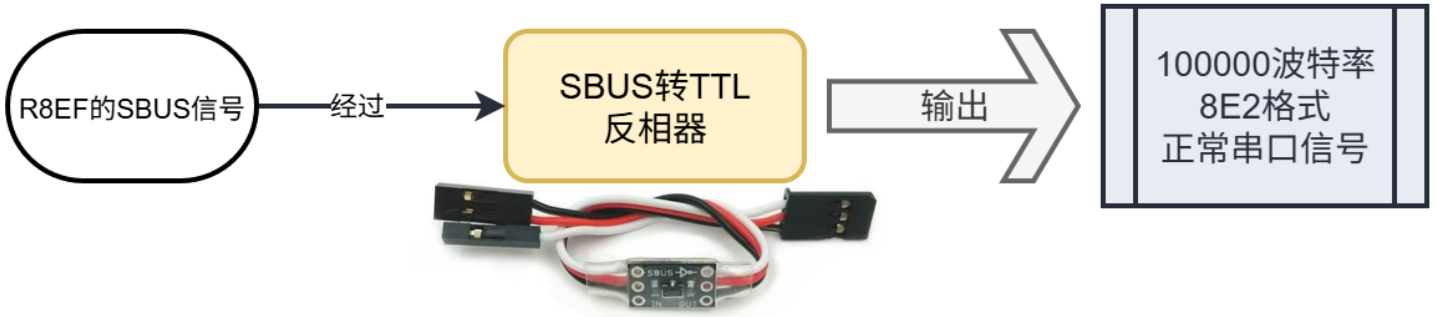
SBUS 协议

- SBUS（Serial Bus）是由日本 FUTABA 公司设计，用于遥控器与接收机、接收机与飞控之间的数字控制信号传输。

- SBUS 协议是 FUTABA 提出的单线数字串行控制协议，采用反向 TTL 逻辑、100 kbps波特率、固定25字节帧结构，可传输16个11位比例通道与2个数字通道。它基于UART（1起始位、8数据位、偶校验、2停止位），使用0x0F作为帧起始、0x00作为结束符，并广泛应用于无人机、航模和机器人控制。

SBUS 协议

- SBUS（Serial Bus）是由日本 FUTABA 公司设计，用于遥控器与接收机、接收机与飞控之间的数字控制信号传输。
- SBUS 协议是 FUTABA 提出的单线数字串行控制协议，采用反向 TTL 逻辑、100 kbps波特率、固定25字节帧结构，可传输16个11位比例通道与2个数字通道。它基于UART（1起始位、8数据位、偶校验、2停止位），使用0x0F作为帧起始、0x00作为结束符，并广泛应用于无人机、航模和机器人控制。



挑战点8：硬件子系统接收并解析遥控器命令





遥控器

发送2.4GHz信号

R8EF接收机

输出SBUS信号

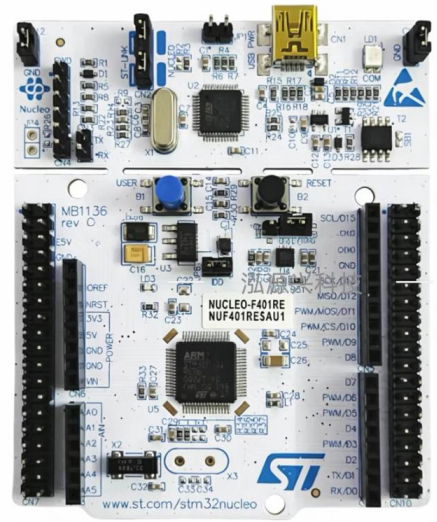
SBUS转TTL
反相器

输出USART信号

飞控芯片

解析指令
根据指令调整自身行为





USART 配置

- 波特率：100,000
- 数据位：8位
- 停止位：2位
- 校验位：1位偶校验
- 模式：输入
- 流控：无

```
    USART_InitTypeDef USART_InitStruct;

    USART_InitStruct.USART_BaudRate            = 100000;
    USART_InitStruct.USART_WordLength          = USART_WordLength_8b;
    USART_InitStruct.USART_StopBits            = USART_StopBits_2;
    USART_InitStruct.USART_Parity              = USART_Parity_Even;
    USART_InitStruct.USART_Mode                = USART_Mode_Rx;
    USART_InitStruct.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
```

数据接收

- 单帧传输时间：100kbps、8E2格式下，每字节占12位（1起始+8数据+1校验+2停止），25字节共300位。传输时间 = 300位 ÷ 100,000bps = 3ms。
- 帧间隔：SBUS高速模式发射频率约141Hz，周期≈1/141≈7ms
- 间隔4ms。



两种数据处理方案

- 方案一

1. 串口每收到一字节数据就触发串口中断；

2. CPU暂停飞控，进入串口中断；

3. CPU自行把USART6的RDR寄存器里的一字节数据搬进内存中

- 方案二

1. 配置DMA，DMA可以自动地将USART6的RDR寄存器里的数据搬运到指定的内存地址中

2. 在此期间，CPU就可以并行地处理自己的飞控

3. 等到合适的时候再用中断来通知CPU去处理



两种数据处理方案

- 方案一

1. 串口每收到一字节数据就触发串口中断；

2. CPU暂停飞控，进入串口中断；

3. CPU自行把USART6的RDR寄存器里的一字节数据搬进内存中

- **方案二**

1. 配置DMA，DMA可以自动地将USART6的RDR寄存器里的数据搬运到指定的内存地址中

2. 在此期间，CPU就可以并行地处理自己的飞控

3. 等到合适的时候再用中断来通知CPU去处理



5.5.1 DMA 方式的基本概念

1. 定义

DMA（Direct Memory Access）即直接存储器访问，几乎是所有计算机系统都具备的一种重要的 I/O 工作机制，是指这样一种传输控制方式：依靠硬件直接在主存与外围设备之间进行数据传输，在数据的 I/O 传输过程中不需要 CPU 执行程序来干预。

直接存储器访问（DMA）意味着在主存储器与 I/O 设备之间有直接的数据传输通路，不必经过 CPU，也称为数据直传。也就是说，输入设备的数据可经系统数据总线直接输入到主存储器；而主存中的数据可经数据总线直接输出给输出设备，所以也称为直接存储器存取。定义的又一层含义是，这样的数据直传是直接由硬件控制实现的，不依靠执行程序指令来实现，所以在 DMA 传输期间不需要 CPU 执行程序来控制干预。

Figure 22. DMA block diagram

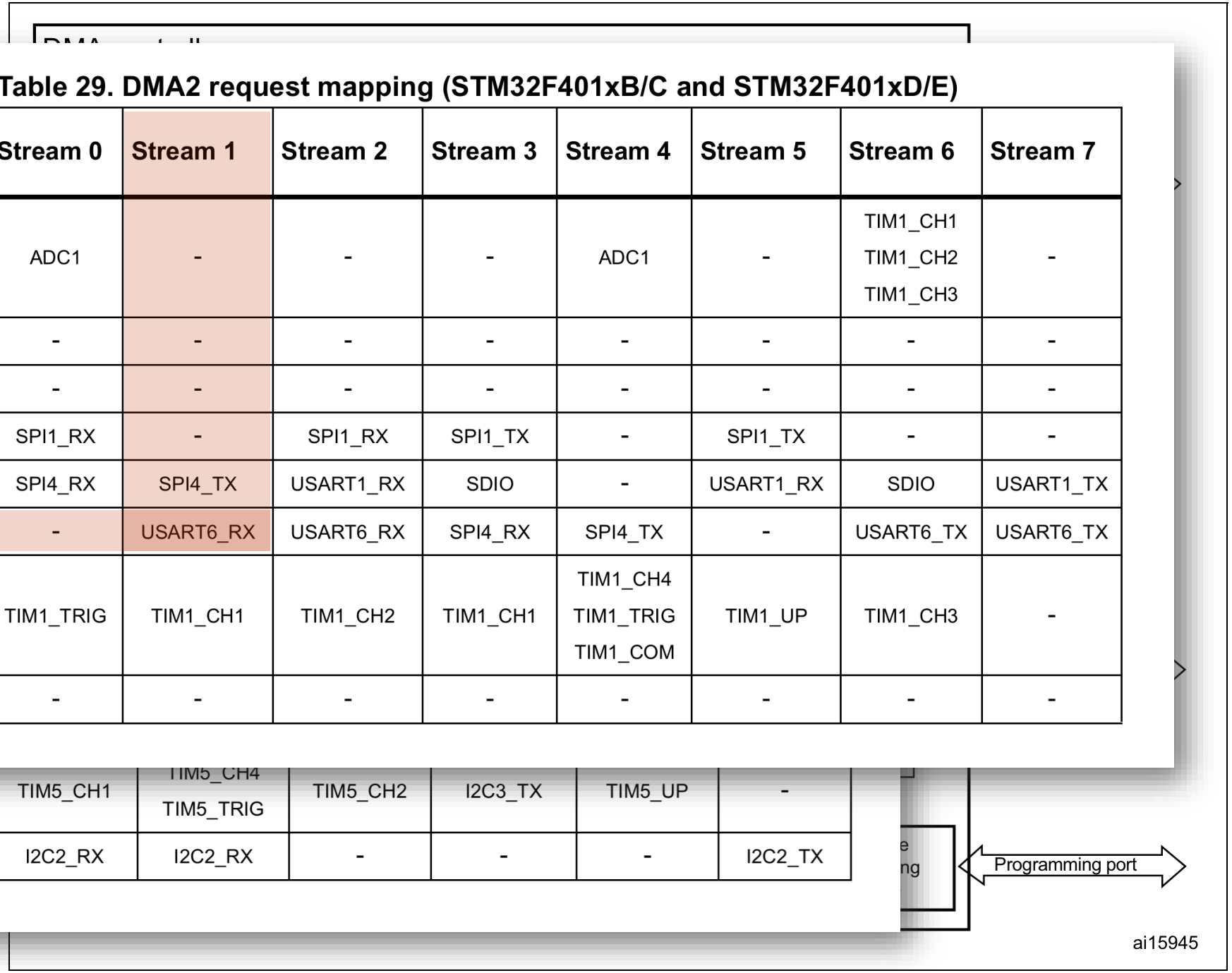


Table 29. DMA2 request mapping (STM32F401xB/C and STM32F401xD/E)

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	ADC1	-	-	-	ADC1	-	TIM1_CH1 TIM1_CH2 TIM1_CH3	-
Channel 1	-	-	-	-	-	-	-	-
Channel 2	-	-	-	-	-	-	-	-
Channel 3	SPI1_RX	-	SPI1_RX	SPI1_TX	-	SPI1_TX	-	-
Channel 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO	-	USART1_RX	SDIO	USART1_TX
Channel 5	-	USART6_RX	USART6_RX	SPI4_RX	SPI4_TX	-	USART6_TX	USART6_TX
Channel 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
Channel 7	-	-	-	-	-	-	-	-

DMA

外设地址寄存器

9.5.7 DMA stream x peripheral address register (DMA_SxPAR) (x = 0..7)

Address offset: 0x18 + 0x18 × *stream number*

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

数据的**源地址**，比如USART6->DR。

将USART_CR3→Bit 6:DMAR位置为1，就相当于打开了USART6向DMA控制器发送“接收请求”的**硬件开关**。每当USART6收到一个完整字节（RXNE标志置位），硬件就会自动触发一次DMA请求，DMA将数据搬运走。

DMA

9.5.8 DMA stream x memory 0 address register (DMA_SxM0AR) (x = 0..7)

Address offset: 0x1C + 0x18 × *stream number*

Reset value: 0x0000 0000

存储器0地址寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- 数据的**目的地址**

9.5.9 DMA stream x memory 1 address register (DMA_SxM1AR) (x = 0..7)

Address offset: 0x20 + 0x18 × *stream number*

Reset value: 0x0000 0000

存储器1地址寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M1A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M1A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- 数据的**备用目的地址**
- 只在“双缓冲模式（Double Buffer Mode）”下生效，用于交替写入两块内存，防止数据覆盖。

DMA

- TCIF：传输完成
- HTIF：半传输
- TEIF：传输错误
- DMEIF：直接模式错误
- FEIF：FIFO错误

9.5.1 DMA low interrupt status register (DMA_LISR) 低中断状态寄存器

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserved	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserved	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF1	HTIF1	TEIF1	DMEIF1	Reserved	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserved	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

9.5.2 DMA high interrupt status register (DMA_HISR) 高中断状态寄存器

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF7	HTIF7	TEIF7	DMEIF7	Reserved	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Reserved	FEIF6
				r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF5	HTIF5	TEIF5	DMEIF5	Reserved	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Reserved	FEIF4
				r	r	r	r		r	r	r	r	r		r

9.4 DMA interrupts

For each DMA stream, an interrupt can be produced on the following events:

- Half-transfer reached
- Transfer complete
- Transfer error
- Fifo error (overrun, underrun or FIFO level error)
- Direct mode error

Separate interrupt enable control bits are available for flexibility as shown in *Table 36*.

Table 36. DMA interrupt requests

Interrupt event	Event flag	Enable control bit
Half-transfer	HTIF	HTIE
Transfer complete	TCIF	TCIE
Transfer error	TEIF	TEIE
FIFO overrun/underrun	FEIF	FEIE
Direct mode error	DMEIF	DMEIE

Note: Before setting an Enable control bit to ‘1’, the corresponding event flag should be cleared, otherwise an interrupt is immediately generated.

DMA

- 对应位置写1
  即可清除
  对应位置的
  中断状态寄存器
- 否则，
  若不清除中断状态
  就会反复进入中断

9.5.3 DMA low interrupt flag clear register (DMA_LIFCR)

Address offset: 0x08

Reset value: 0x0000 0000

低中断标志清除寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
				w	w	w	w		w	w	w	w	w		w

9.5.4 DMA high interrupt flag clear register (DMA_HIFCR)

Address offset: 0x0C

Reset value: 0x0000 0000

高中断标志清除寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Reserved	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Reserved	CFEIF6
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Reserved	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Reserved	CFEIF4
				w	w	w	w		w	w	w	w	w		w

DMA

数据流配置寄存器

9.5.5 DMA stream x configuration register (DMA_SxCR) (x = 0..7)

This register is used to configure the concerned stream.

Address offset: 0x10 + 0x18 × *stream number*

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bit 0 **EN**: Stream enable / flag stream ready when read low

This bit is set and cleared by software.

0: Stream disabled

1: Stream enabled

This bit may be cleared by hardware:

- on a DMA end of transfer (stream ready to be configured)
- if a transfer error occurs on the AHB master buses
- when the FIFO threshold on memory AHB port is not compatible with the size of the burst

When this bit is read as 0, the software is allowed to program the Configuration and FIFO bits registers. It is forbidden to write these registers when the EN bit is read as 1.

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 4 **TCIE**: Transfer complete interrupt enable
This bit is set and cleared by software.
0: TC interrupt disabled
1: TC interrupt enabled

Bit 3 **HTIE**: Half transfer interrupt enable
This bit is set and cleared by software.
0: HT interrupt disabled
1: HT interrupt enabled

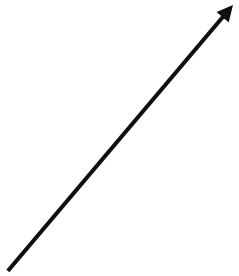
Bit 2 **TEIE**: Transfer error interrupt enable
This bit is set and cleared by software.
0: TE interrupt disabled
1: TE interrupt enabled

Bit 1 **DMEIE**: Direct mode error interrupt enable
This bit is set and cleared by software.
0: DME interrupt disabled
1: DME interrupt enabled

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bit 5 **PFCTRL**: Peripheral flow controller

This bit is set and cleared by software.

0: The DMA is the flow controller

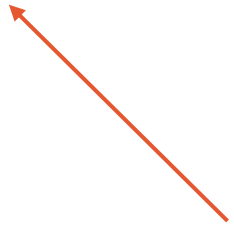
1: The peripheral is the flow controller

This bit is protected and can be written only if EN is ‘0’.

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bits 7:6 **DIR[1:0]**: Data transfer direction

These bits are set and cleared by software.

00: Peripheral-to-memory

01: Memory-to-peripheral

10: Memory-to-memory

11: reserved

These bits are protected and can be written only if EN is ‘0’.

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bit 8 **CIRC**: Circular mode

This bit is set and cleared by software and can be cleared by hardware.

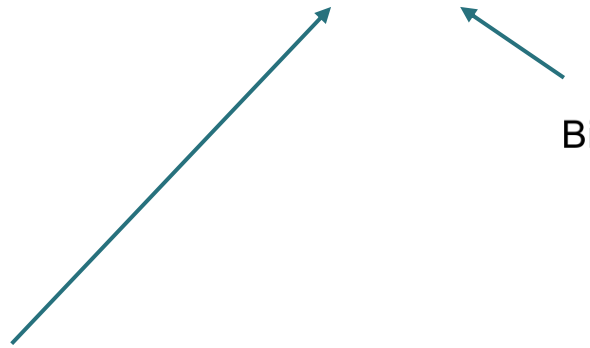
0: Circular mode disabled

1: Circular mode enabled

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bit 9 **PINC**: Peripheral increment mode

This bit is set and cleared by software.

0: Peripheral address pointer is fixed

1: Peripheral address pointer is incremented after each data transfer (increment is done according to PSIZE)

This bit is protected and can be written only if EN is ‘0’.

Bit 10 **MINC**: Memory increment mode

This bit is set and cleared by software.

0: Memory address pointer is fixed

1: Memory address pointer is incremented after each data transfer (increment is done according to MSIZE)

This bit is protected and can be written only if EN is ‘0’.

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 14:13 **MSIZE[1:0]**: Memory data size

These bits are set and cleared by software.

00: byte (8-bit)

01: half-word (16-bit)

10: word (32-bit)

11: reserved

These bits are protected and can be written only if EN is ‘0’.

Bits 12:11 **PSIZE[1:0]**: Peripheral data size

These bits are set and cleared by software.

00: Byte (8-bit)

01: Half-word (16-bit)

10: Word (32-bit)

11: reserved

These bits are protected and can be written only if EN is ‘0’

DMA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **PINCOS**: Peripheral increment offset size

This bit is set and cleared by software

0: The offset size for the peripheral address calculation is linked to the PSIZE

1: The offset size for the peripheral address calculation is fixed to 4 (32-bit alignment).

This bit has no meaning if bit PINC = '0'.

This bit is protected and can be written only if EN = '0'.

This bit is forced low by hardware when the stream is enabled (bit EN = '1') if the direct mode is selected or if PBURST are different from “00”.

DMA

Bits 17:16 **PL[1:0]**: Priority level

These bits are set and cleared by software.

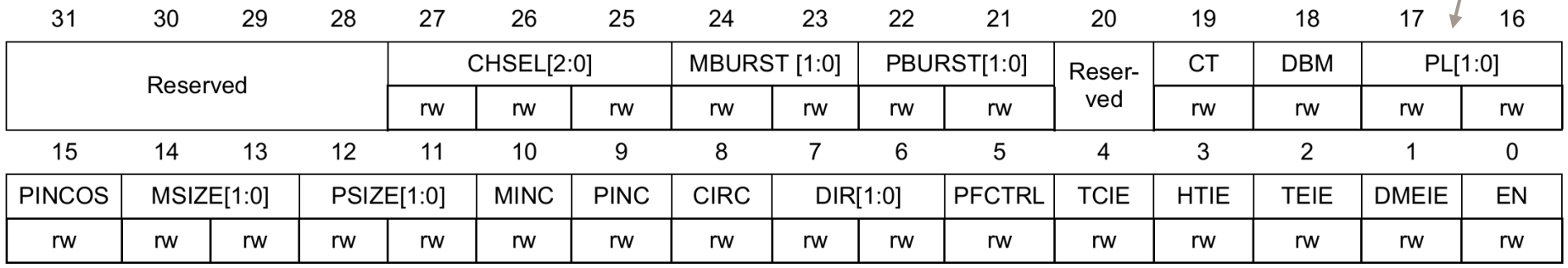
00: Low

01: Medium

10: High

11: Very high

These bits are protected and can be written only if EN is ‘0’.



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMA

Bit 19 **CT**: Current target (only in double buffer mode)

This bits is set and cleared by hardware. It can also be written by software.

0: The current target memory is Memory 0 (addressed by the DMA_SxM0AR pointer)

1: The current target memory is Memory 1 (addressed by the DMA_SxM1AR pointer)

Bit 18 **DBM**: Double buffer mode

This bits is set and cleared by software.

0: No buffer switching at the end of transfer

1: Memory target switched at the end of the DMA transfer

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMA

These bits are protected and can be written only if EN is ‘0’
In direct mode, these bits are forced to 0x0 by hardware as soon as bit EN= '1'.

Bits 24:23 **MBURST**: Memory burst transfer configuration

These bits are set and cleared by software.
00: single transfer
01: INCR4 (incremental burst of 4 beats)
10: INCR8 (incremental burst of 8 beats)
11: INCR16 (incremental burst of 16 beats)

Bits 22:21 **PBURST[1:0]**: Peripheral burst transfer configuration

These bits are set and cleared by software.
00: single transfer
01: INCR4 (incremental burst of 4 beats)
10: INCR8 (incremental burst of 8 beats)
11: INCR16 (incremental burst of 16 beats)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

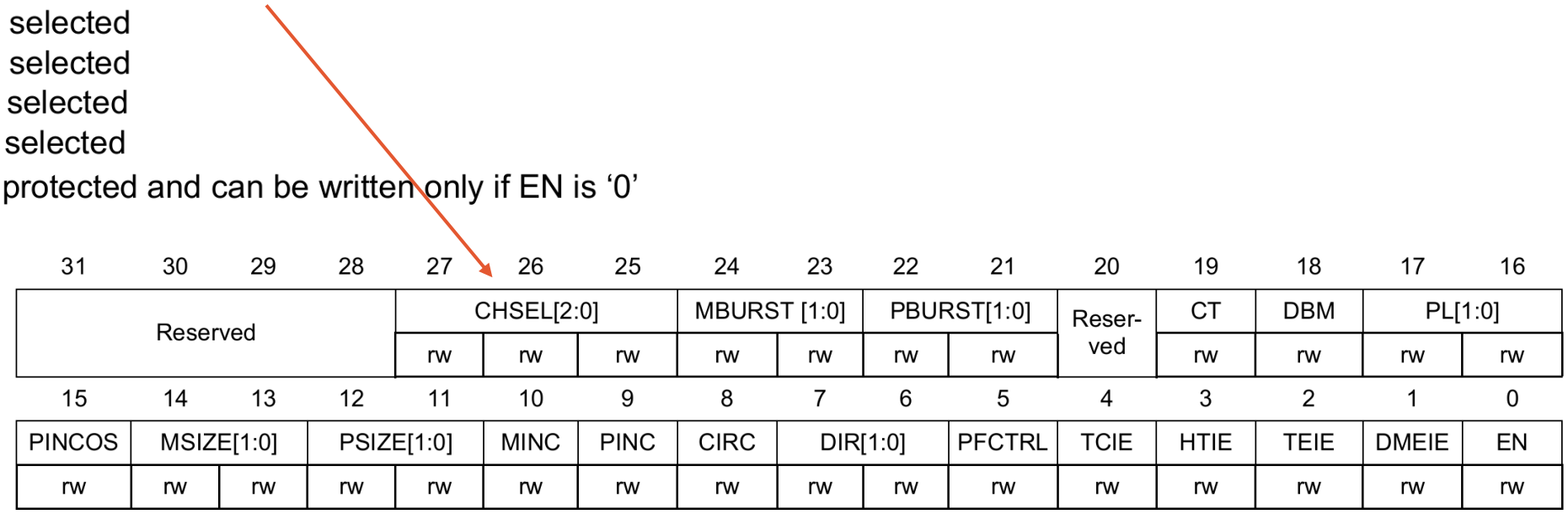
DMA

Bits 27:25 **CHSEL[2:0]**: Channel selection

These bits are set and cleared by software.

000: channel 0 selected
001: channel 1 selected
010: channel 2 selected
011: channel 3 selected
100: channel 4 selected
101: channel 5 selected
110: channel 6 selected
111: channel 7 selected

These bits are protected and can be written only if EN is ‘0’



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMA

数据项数寄存器

9.5.6 DMA stream x number of data register (DMA_SxNDTR) (x = 0..7)

Address offset: 0x14 + 0x18 × *stream number*

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

记录“传输次数（数据项个数）”

DMA 的每一次搬运动作，统称为一次 “Transfer（传输）”。只要发生了一次 Transfer，硬件就会把 NDTR 减 1。

DMA

FIFO控制寄存器

9.5.10 DMA stream x FIFO control register (DMA_SxFCR) (x = 0..7)

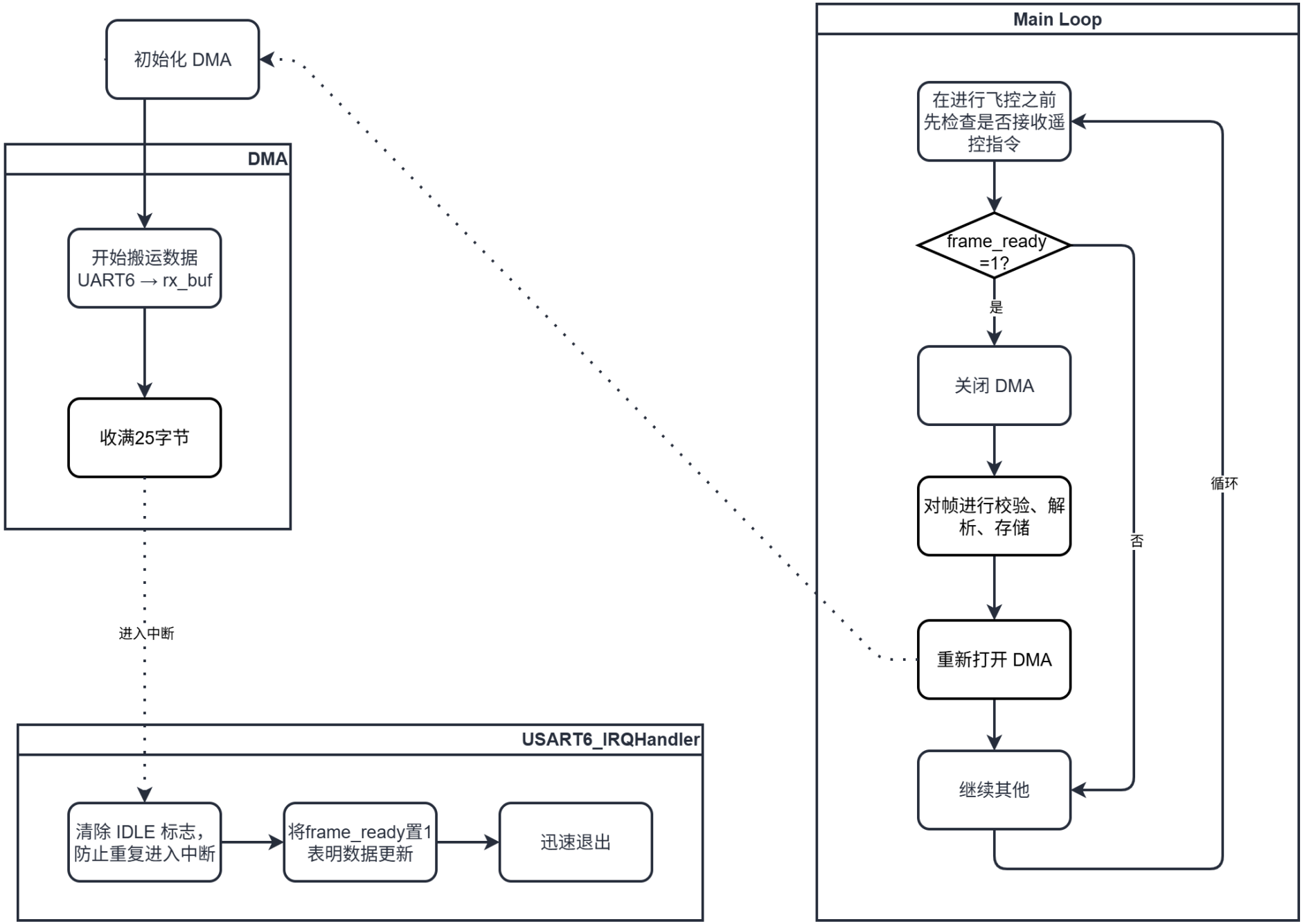
Address offset: 0x24 + 0x18 × *stream number*

Reset value: 0x0000 0021

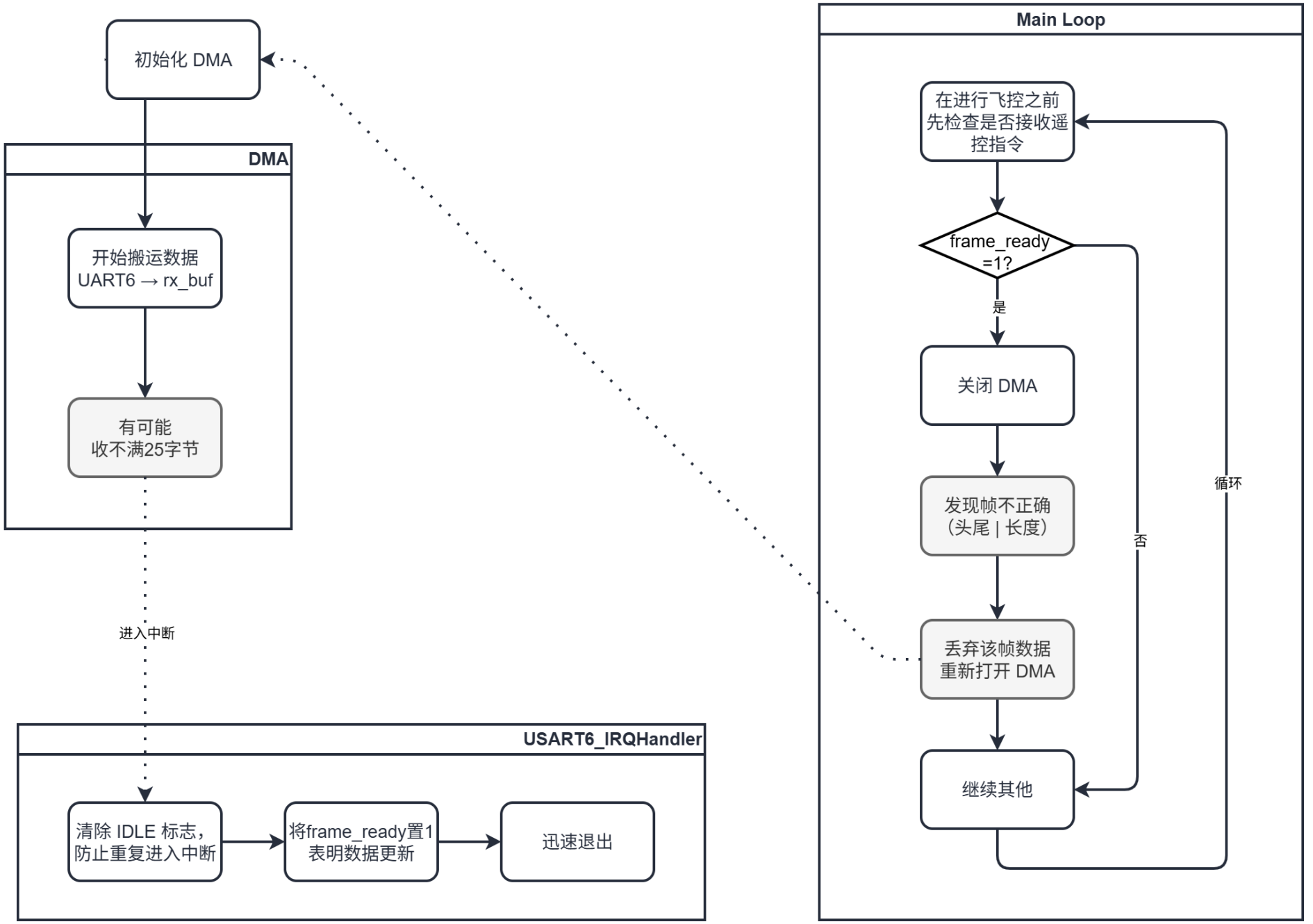
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								FEIE	Reserved	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

STM32 的每个 DMA 数据流（Stream）内部都有一个 4 **字深（**4 × 32**位** = 16 **字节）**的 FIFO 缓冲区。
如果开启了 FIFO，外设发来的数据先暂存在 FIFO 池子里，我们可以设置一个阈值（1/4 满、1/2 满、全满），数据量达到阈值时，DMA 再一次性向总线申请权限，把这些数据打包，一口气搬进内存。

- 软件流程

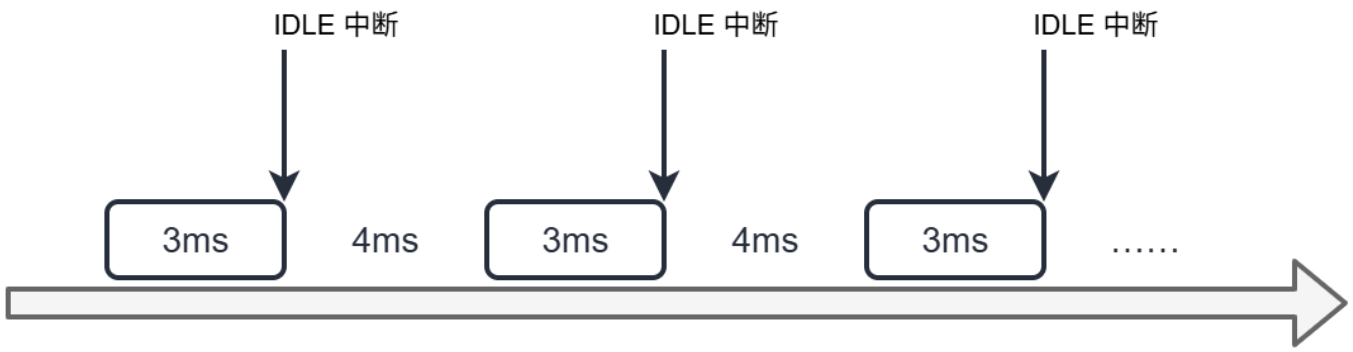

初始化 DMA
DMA
开始搬运数据
UART6 → rx_buf
收满25字节
进入中断
USART6_IRQHandler
清除 IDLE 标志，
防止重复进入中断
将frame_ready置1
表明数据更新
迅速退出
Main Loop
在进行飞控之前
先检查是否接收遥
控指令
frame_ready
=1?
是
否
关闭 DMA
对帧进行校验、解
析、存储
重新打开 DMA
继续其他
循环

- 软件流程


初始化 DMA
DMA
开始搬运数据
UART6 → rx_buf
有可能
收不满25字节
进入中断
USART6_IRQHandler
清除 IDLE 标志，
防止重复进入中断
将frame_ready置1
表明数据更新
迅速退出
Main Loop
在进行飞控之前
先检查是否接收遥
控指令
frame_ready
=1?
是
否
关闭 DMA
发现帧不正确
（头尾 | 长度）
丢弃该帧数据
重新打开 DMA
继续其他
循环

IDLE 中断

- 在 STM32 的 USART 外设中，IDLE 代表“总线空闲状态”。
- SBUS 接收机发送一帧 25 个字节，这 25 个字节是紧密相连、中间毫无停顿地发出来的，大约需 3ms。



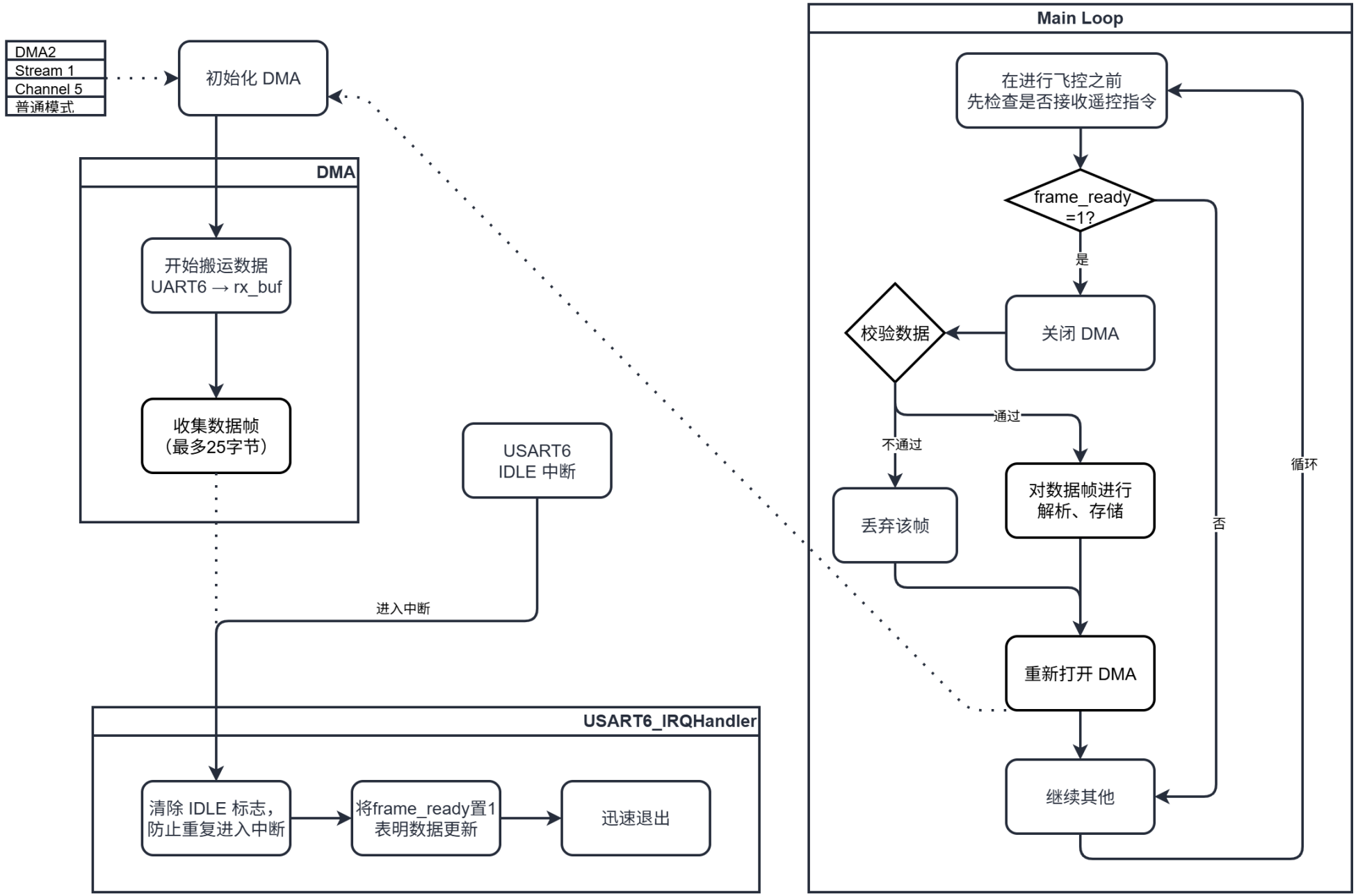
- 在这 3ms 期间：总线上一直有数据在翻转，绝对不可能触发 IDLE。
- 在第 25 个字节发完的瞬间：最后一个停止位结束，总线恢复高电平。只要总线保持高电平超过 1 个字节的时间（约 120μs），IDLE 中断就会立刻触发。

DMA配置清单

1. USART6接收SBUS数据所使用的DMA资源为：DMA2，Stream 1，Channel 5。
2. 寄存器配置清单如下：

寄存器名称	寄存器地址	写入值	含义
DMA2数据流1配置寄存器	0x40026428	0x0001940D	通道5、数据从外设搬到内存、存储器地址递增、外设地址固定、每次搬运1字节、普通模式、最后一位置1使能数据流
DMA2数据流1数据项数寄存器	0x4002642C	0x00000019	NDTR=25，设定本次传输总共搬运25个数据项
DMA2数据流1外设地址寄存器	0x40026430	0x40011404	PAR指向USART6数据寄存器USART6_DR的物理地址，作为DMA传输的固定源地址
DMA2数据流1 存储器0地址寄存器	0x40026434	rx_buf	指向我们自行开辟的接收缓冲区的首地址，作为DMA传输的目的地址
USART6控制寄存器3	0x40011408	0x00000040	DMAR=1，置位第6位，使能USART6的DMA接收请求，打通串口向DMA发送请求的硬件链路

- 最终流程


DMA2
Stream 1
Channel 5
普通模式
初始化 DMA
DMA
开始搬运数据
UART6 → rx_buf
收集数据帧
（最多25字节）
USART6
IDLE 中断
进入中断
USART6_IRQHandler
清除 IDLE 标志，
防止重复进入中断
将frame_ready置1
表明数据更新
迅速退出
Main Loop
在进行飞控之前
先检查是否接收遥控指令
frame_ready
=1?
是
否
关闭 DMA
校验数据
通过
不通过
丢弃该帧
对数据帧进行
解析、存储
重新打开 DMA
继续其他
循环

文件结构总览

start.s

堆栈配置
中断向量表
- 复位入口
- 默认中断处理

main.s

参数定义
内存分配
外设初始化
主循环
中断处理
数据拼接

START.S

- 第一步 —— 堆栈内存分配

- Cortex-M4 的栈是"满递减"： SP 初值 = 栈区最高地址，压栈时 SP 先减。

start.s | main.s | startup.s

```
1  ; 第一步：堆栈分配
2  Stack_Size  EQU     0x00000800  ; 栈分配2kb（目前不需要堆，不进行定义）
3
4  ; 栈段
5      AREA    STACK, NOINIT, READWRITE, ALIGN=3
6  Stack_Mem   SPACE   Stack_Size  ; 为栈分配内存
7  __initial_sp    ; 分配的内存后面紧接标号，即标记出栈顶位置：Cortex-M的栈是“满递减”，SP从高地址向下生长
8
```

START.S

- 第二步 —— 中断向量表

- 汇编点灯时 →

```
6   __Vectors
7       DCD     __initial_sp
8       DCD     Reset_Handler
9
```

- 但是现在，SBUS数据的接收还需要用到一些中断函数，
  （如 USART6 的 IDLE 中断会进入 USART6_IRQHandler 进行中断处理）
- 中断函数的入口就需要在中断向量表中定义。

START.S

- 硬件是按固定地址顺序来识别的，**完全不关心名字。**
- 在汇编启动文件里出现 USART6_IRQHandler 这个名字，只是因为链接器需要它来对应正确的地址槽位。

- Cortex-M4 内核的 NVIC（嵌套向量中断控制器）在响应一个中断（IRQ）时，完全依据一个硬编码的“中断号（IRQ Number）”。
- 这个中断号会在向量表中被转换成一个固定的地址偏移。

START.S

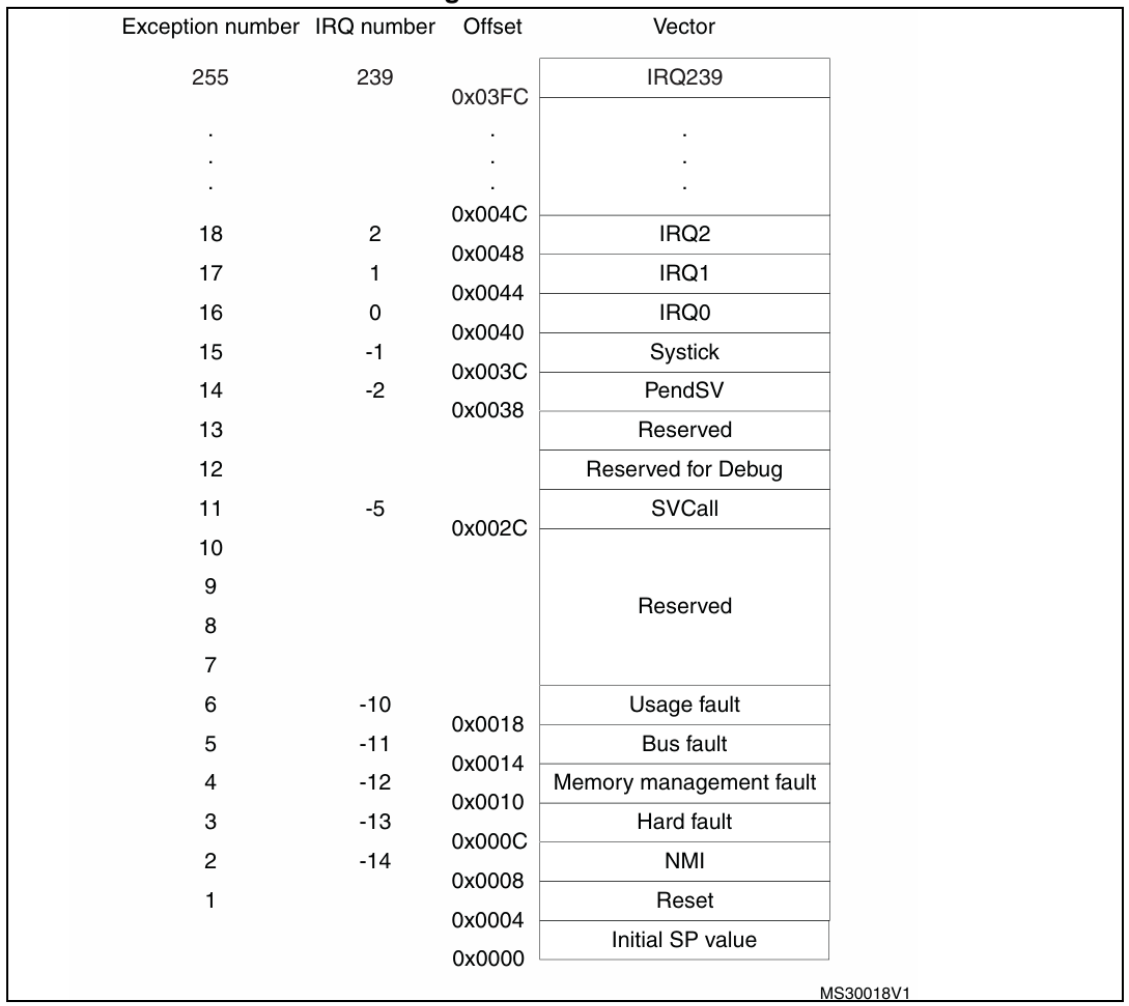
- 手册里的“ 240个 ”是 ARM Cortex-M4 内核架构上允许的最大外部中断数，但这并不意味着芯片厂商会把这些位置全部实现。具体要填多少个，完全取决于芯片本身实现了多少中断。

- 对于 STM32F401RET6，答案很明确：只需要填充 101 个向量（16个内核 + 85个外设），而不是240个。

2.3.4 Vector table

The vector table contains the reset value of the stack pointer, and the start addresses, also called exception vectors, for all exception handlers. *Figure 11 on page 39* shows the order of the exception vectors in the vector table. The least-significant bit of each vector must be 1, indicating that the exception handler is Thumb code.

Figure 11. Vector table



On system reset, the vector table is fixed at address 0x00000000. Privileged software can write to the VTOR to relocate the vector table start address to a different memory location, in the range 0x00000080 to 0x3FFFFF80, see *Vector table offset register (VTOR) on page 212*.

Table 38. Vector table for STM32F401xB/CSTM32F401xD/E

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-2	fixed	NMI	Non maskable interrupt, Clock Security System	0x0000 0008
-	-1	fixed	HardFault	All class of fault	0x0000 000C
-	0	settable	MemManage	Memory management	0x0000 0010
-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 002B
-	3	settable	SVCall	System Service call via SWI instruction	0x0000 002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000 0030
-		-	-	Reserved	0x0000 0034
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	Systick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	EXTI16 / PVD	EXTI Line 16 interrupt / PVD through EXTI line detection interrupt	0x0000 0044
2	9	settable	EXTI21 / TAMP_STAMP	EXTI Line 21 interrupt / Tamper and TimeStamp interrupts through the EXTI line	0x0000 0048
3	10	settable	EXTI22 / RTC_WKUP	EXTI Line 22 interrupt / RTC Wake-up interrupt through the EXTI line	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_Stream0	DMA1 Stream0 global interrupt	0x0000 006C

Table 38. Vector table for STM32F401xB/CSTM32F401xD/E (continued)

Position	Priority	Type of priority	Acronym	Description	Address
12	19	settable	DMA1_Stream1	DMA1 Stream1 global interrupt	0x0000 0070
13	20	settable	DMA1_Stream2	DMA1 Stream2 global interrupt	0x0000 0074
14	21	settable	DMA1_Stream3	DMA1 Stream3 global interrupt	0x0000 0078
15	22	settable	DMA1_Stream4	DMA1 Stream4 global interrupt	0x0000 007C
16	23	settable	DMA1_Stream5	DMA1 Stream5 global interrupt	0x0000 0080
17	24	settable	DMA1_Stream6	DMA1 Stream6 global interrupt	0x0000 0084
18	25	settable	ADC	ADC1 global interrupts	0x0000 0088
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
24	31	settable	TIM1_BRK_TIM9	TIM1 Break interrupt and TIM9 global interrupt	0x0000 00A0
25	32	settable	TIM1_UP_TIM10	TIM1 Update interrupt and TIM10 global interrupt	0x0000 00A4
26	33	settable	TIM1_TRG_COM_TIM11	TIM1 Trigger and Commutation interrupts and TIM11 global interrupt	0x0000 00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000 00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000 00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000 00B8
31	38	settable	I2C1_EV	I^2C1 event interrupt	0x0000 00BC
32	39	settable	I2C1_ER	I^2C1 error interrupt	0x0000 00C0
33	40	settable	I2C2_EV	I^2C2 event interrupt	0x0000 00C4
34	41	settable	I2C2_ER	I^2C2 error interrupt	0x0000 00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000 00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000 00D0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0
41	48	settable	EXTI17 / RTC_Alarm	EXTI Line 17 interrupt / RTC Alarms (A and B) through EXTI line interrupt	0x0000 00E4
42	49	settable	EXTI18 / OTG_FS_WKUP	EXTI Line 18 interrupt / USB On-The-Go FS Wake-up through EXTI line interrupt	0x0000 00E8
47	54	settable	DMA1_Stream7	DMA1 Stream7 global interrupt	0x0000 00FC

Table 38. Vector table for STM32F401xB/CSTM32F401xD/E (continued)

Position	Priority	Type of priority	Acronym	Description	Address
49	56	settable	SDIO	SDIO global interrupt	0x0000 0104
50	57	settable	TIM5	TIM5 global interrupt	0x0000 0108
51	58	settable	SPI3	SPI3 global interrupt	0x0000 010C
56	63	settable	DMA2_Stream0	DMA2 Stream0 global interrupt	0x0000 0120
57	64	settable	DMA2_Stream1	DMA2 Stream1 global interrupt	0x0000 0124
58	65	settable	DMA2_Stream2	DMA2 Stream2 global interrupt	0x0000 0128
59	66	settable	DMA2_Stream3	DMA2 Stream3 global interrupt	0x0000 012C
60	67	settable	DMA2_Stream4	DMA2 Stream4 global interrupt	0x0000 0130
67	74	settable	OTG_FS	USB On The Go FS global interrupt	0x0000 014C
68	75	settable	DMA2_Stream5	DMA2 Stream5 global interrupt	0x0000 0150
69	76	settable	DMA2_Stream6	DMA2 Stream6 global interrupt	0x0000 0154
70	77	settable	DMA2_Stream7	DMA2 Stream7 global interrupt	0x0000 0158
71	78	settable	USART6	USART6 global interrupt	0x0000 015C
72	79	settable	I2C3_EV	I^2C3 event interrupt	0x0000 0160
73	80	settable	I2C3_ER	I^2C3 error interrupt	0x0000 0164
81	88	Settable	FPU	FPU global interrupt	0x0000 0184
84	91	settable	SPI4	SPI 4 global interrupt	0x0000 0190

F401RET6
外设所用到
的所有中断

中断向量表 —— 芯片内系统异常部分

```
;----------------------- Cortex-M4 系统异常处理 -------------------------
__Vectors       DCD     __initial_sp                    ; 0x00 初始 SP
                DCD     Reset_Handler                   ; 0x04 Reset
                DCD     NMI_Handler                     ; 0x08
                DCD     HardFault_Handler               ; 0x0C
                DCD     MemManage_Handler               ; 0x10
                DCD     BusFault_Handler                ; 0x14
                DCD     UsageFault_Handler              ; 0x18
                DCD     0                               ; 0x1C 保留
                DCD     0                               ; 0x20 保留
                DCD     0                               ; 0x24 保留
                DCD     0                               ; 0x28 保留
                DCD     SVC_Handler                     ; 0x2C
                DCD     DebugMon_Handler                ; 0x30
                DCD     0                               ; 0x34 保留
                DCD     PendSV_Handler                  ; 0x38
                DCD     SysTick Handler                 ; 0x3C
```

中断向量表——外设中断部分

```
;---------------------- 外部中断 IRQ0 ~ IRQ84 -------------------------
                DCD     Default_Handler             ; 0x40 IRQ0  WWDG
                DCD     Default_Handler             ; 0x44 IRQ1  PVD(EXTI16)
                DCD     Default_Handler             ; 0x48 IRQ2  TAMP_STAMP(EXTI21)
                DCD     Default_Handler             ; 0x4C IRQ3  RTC_WKUP(EXTI22)
                DCD     Default_Handler             ; 0x50 IRQ4  FLASH
                DCD     Default_Handler             ; 0x54 IRQ5  RCC
                DCD     Default_Handler             ; 0x58 IRQ6  EXTI0
                DCD     Default_Handler             ; 0x5C IRQ7  EXTI1
                DCD     Default_Handler             ; 0x60 IRQ8  EXTI2
                DCD     Default_Handler             ; 0x64 IRQ9  EXTI3
                DCD     Default_Handler             ; 0x68 IRQ10 EXTI4
                DCD     Default_Handler             ; 0x6C IRQ11 DMA1_Stream0
                DCD     Default_Handler             ; 0x70 IRQ12 DMA1_Stream1
                DCD     Default_Handler             ; 0x74 IRQ13 DMA1_Stream2
                DCD     Default_Handler             ; 0x78 IRQ14 DMA1_Stream3
                DCD     Default_Handler             ; 0x7C IRQ15 DMA1_Stream4
                DCD     Default_Handler             ; 0x80 IRQ16 DMA1_Stream5
                DCD     Default_Handler             ; 0x84 IRQ17 DMA1_Stream6
                DCD     Default_Handler             ; 0x88 IRQ18 ADC
                DCD     0                           ; 0x8C IRQ19 保留
                DCD     0                           ; 0x90 IRQ20 保留
                DCD     0                           ; 0x94 IRQ21 保留
                DCD     0                           ; 0x98 IRQ22 保留
                DCD     Default_Handler             ; 0x9C IRQ23 EXTI9_5
                DCD     Default_Handler             ; 0xA0 IRQ24 TIM1_BRK_TIM9
                DCD     Default_Handler             ; 0xA4 IRQ25 TIM1_UP_TIM10
                DCD     Default_Handler             ; 0xA8 IRQ26 TIM1_TRG_COM_TIM11
                DCD     Default_Handler             ; 0xAC IRQ27 TIM1_CC
                DCD     Default_Handler             ; 0xB0 IRQ28 TIM2
                DCD     Default_Handler             ; 0xB4 IRQ29 TIM3
                DCD     Default_Handler             ; 0xB8 IRQ30 TIM4
                DCD     Default_Handler             ; 0xBC IRQ31 I2C1_EV
                DCD     Default_Handler             ; 0xC0 IRQ32 I2C1_ER
                DCD     Default_Handler             ; 0xC4 IRQ33 I2C2_EV
                DCD     Default_Handler             ; 0xC8 IRQ34 I2C2_ER
                DCD     Default_Handler             ; 0xCC IRQ35 SPI1
                DCD     Default_Handler             ; 0xD0 IRQ36 SPI2
                DCD     Default_Handler             ; 0xD4 IRQ37 USART1
                DCD     USART2_IRQHandler           ; 0xD8 IRQ38 USART2（现在只做发送，不用中断）
                DCD     0                           ; 0xDC IRQ39 保留
                DCD     Default_Handler             ; 0xE0 IRQ40 EXTI15_10
                DCD     Default_Handler             ; 0xE4 IRQ41 RTC Alarm(EXTI17)
                DCD     Default_Handler             ; 0xE8 IRQ42 OTG_FS_WKUP(EXTI18)
                DCD     0                           ; 0xEC IRQ43 保留
                DCD     0                           ; 0xF0 IRQ44 保留
                DCD     0                           ; 0xF4 IRQ45 保留
                DCD     0                           ; 0xF8 IRQ46 保留
                DCD     Default_Handler             ; 0xFC IRQ47 DMA1_Stream7
                DCD     0                           ; 0x100 IRQ48 保留
                DCD     Default_Handler             ; 0x104 IRQ49 SDIO
                DCD     Default_Handler             ; 0x108 IRQ50 TIM5
                DCD     Default_Handler             ; 0x10C IRQ51 SPI3
                DCD     0                           ; 0x110 IRQ52 保留
                DCD     0                           ; 0x114 IRQ53 保留
                DCD     0                           ; 0x118 IRQ54 保留
                DCD     0                           ; 0x11C IRQ55 保留
                DCD     Default_Handler             ; 0x120 IRQ56 DMA2_Stream0
                DCD     Default_Handler             ; 0x124 IRQ57 DMA2_Stream1
                DCD     Default_Handler             ; 0x128 IRQ58 DMA2_Stream2
                DCD     Default_Handler             ; 0x12C IRQ59 DMA2_Stream3
                DCD     Default_Handler             ; 0x130 IRQ60 DMA2_Stream4
                DCD     0                           ; 0x134 IRQ61 保留
                DCD     0                           ; 0x138 IRQ62 保留
                DCD     0                           ; 0x13C IRQ63 保留
                DCD     0                           ; 0x140 IRQ64 保留
                DCD     0                           ; 0x144 IRQ65 保留
                DCD     0                           ; 0x148 IRQ66 保留
                DCD     Default_Handler             ; 0x14C IRQ67 OTG_FS
                DCD     Default_Handler             ; 0x150 IRQ68 DMA2_Stream5
                DCD     Default_Handler             ; 0x154 IRQ69 DMA2_Stream6
                DCD     Default_Handler             ; 0x158 IRQ70 DMA2_Stream7
                DCD     USART6_IRQHandler           ; 0x15C IRQ71 USART6 ★SBUS 接收用这个
                DCD     Default_Handler             ; 0x160 IRQ72 I2C3_EV
                DCD     Default_Handler             ; 0x164 IRQ73 I2C3_ER
                DCD     0                           ; 0x168 IRQ74 保留
                DCD     0                           ; 0x16C IRQ75 保留
                DCD     0                           ; 0x170 IRQ76 保留
                DCD     0                           ; 0x174 IRQ77 保留
                DCD     0                           ; 0x178 IRQ78 保留
                DCD     0                           ; 0x17C IRQ79 保留
                DCD     0                           ; 0x180 IRQ80 保留
                DCD     Default_Handler             ; 0x184 IRQ81 FPU
                DCD     0                           ; 0x188 IRQ82 保留
                DCD     0                           ; 0x18C IRQ83 保留
                DCD     Default_Handler             ; 0x190 IRQ84 SPI4
```

中断向量表 —— 默认中断处理

```
253  Default_Handler
254                  B       Default_Handler
255
256
257  NMI_Handler
258  HardFault_Handler
259  MemManage_Handler
260  BusFault_Handler
261  UsageFault_Handler
262  SVC_Handler
263  DebugMon_Handler
264  PendSV_Handler
265  SysTick_Handler
266  USART2_IRQHandler
267  USART6_IRQHandler
268                  B       Default_Handler
269
270                  END
271
```

- 至此，
  启动文件编写完毕。

MAIN.S

- 准备工作
- 第一步 —— 参数定义

- 先保存一些要用到的寄存器的地址值
- RCC时钟、GPIO、USART串口、DMA、NVIC中断。

```
77  ;================================================================
78  ;  0. 寄存器地址表
79  ;     基地址出处：RM0368 P38 / P39  Table 1
80  ;                 "STM32F401xB/C and STM32F401xD/E register boundary addresses"
81  ;================================================================
82  ; ---- 系统控制 ----
83  FLASH_ACR       EQU     0x40023C00      ; RM0368 P38 边界表 / P60 寄存器说明
84  PWR_CR          EQU     0x40007000      ; RM0368 P39 边界表 / P87 寄存器说明
85  RCC_CR          EQU     0x40023800      ; RM0368 P38 / P103
86  RCC_PLLCFGR     EQU     0x40023804      ; RM0368 P105
87  RCC_CFGR        EQU     0x40023808      ; RM0368 P107（SW/SWS 位定义在 P109）
88  RCC_AHB1ENR     EQU     0x40023830      ; RM0368 P118
89  RCC_APB1ENR     EQU     0x40023840      ; RM0368 P119（USART2EN/PWREN 在 P120）
90  RCC_APB2ENR     EQU     0x40023844      ; RM0368 P122（USART6EN=bit5, P123）
91
92  ; ---- GPIOC（基地址 0x4002 0800, RM0368 P38）----
93  GPIOC_MODER     EQU     0x40020800      ; RM0368 P158
94  GPIOC_OSPEEDR   EQU     0x40020808      ; RM0368 P159
95  GPIOC_PUPDR     EQU     0x4002080C      ; RM0368 P160
96  GPIOC_AFRL      EQU     0x40020820      ; RM0368 P162（PC7 的 AF 在 bit31:28）
97
98  ; ---- USART6（基地址 0x4001 1400, RM0368 P39）— 收 SBUS ----
99  ;      寄存器偏移与 USART2 完全相同，只是基地址不同
100 USART6_SR       EQU     0x40011400      ; RM0368 P548（IDLE=bit4, P549）
101 USART6_DR       EQU     0x40011404      ; RM0368 P551
102 USART6_BRR      EQU     0x40011408      ; RM0368 P551
103 USART6_CR1      EQU     0x4001140C      ; RM0368 P551（各使能位 P552~P553）
104 USART6_CR2      EQU     0x40011410      ; RM0368 P554（STOP[1:0]=bit13:12）
105 USART6_CR3      EQU     0x40011414      ; RM0368 P555（DMAR=bit6, P556）
106
107 ; ---- DMA2（基地址 0x4002 6400, RM0368 P38）----
108 ;      Stream1 的寄存器偏移 = 0x10 + 0x18 × 1 = 0x28（RM0368 P191）
109 DMA2_LISR        EQU     0x40026400      ; RM0368 P188
110 DMA2_LIFCR       EQU     0x40026408      ; RM0368 P190
111 DMA2_S1CR        EQU     0x40026428      ; RM0368 P191
112 DMA2_S1NDTR      EQU     0x4002642C      ; RM0368 P194
113 DMA2_S1PAR       EQU     0x40026430      ; RM0368 P195
114 DMA2_S1M0AR      EQU     0x40026434      ; RM0368 P195
115 DMA2_S1FCR       EQU     0x4002643C      ; RM0368 P196
116
117 ; ---- NVIC ----
118 ;      IRQ71(USART6) → ISER2(0xE000E108) bit7          PM0214 P195
119 ;      IRQ71(USART6) → IPR17(0xE000E444) bit31:24      PM0214 P200
120 NVIC_ISER2      EQU     0xE000E108
121 NVIC_IPR17      EQU     0xE000E444
122
```

MAIN.S

- 准备工作
- 第二步 —— 内存开辟

- DMA 搬运过来的数据
  放在 rx_buf 里；
- 解析完的数据
  放在 channels 里；
- 数据更新标志 frame_ready 。

```
145                 AREA    SBUS_RAM, NOINIT, READWRITE, ALIGN=3
146
147 rx_buf          SPACE   SBUS_FRAME_LEN ; ★ DMA 唯一的落点，只有 25 字节
148                                         ;   收满 25 个 DMA 就自动停，
149                                         ;   不会有"写过头"这回事
150                 ALIGN   4               ; 25 不是 4 的倍数，补 3 个字节，
151                                         ; 让后面的变量仍然按字（4 字节）对齐
152 channels        SPACE   64              ; 16 通道 × 4 字节，解析结果
153 frame_ready     SPACE   4               ; ISR 置 1：rx_buf 里有一帧完整的 25 字节
154
```

MAIN.S —— 主函数流程

```
216  asm_main
217                  BL      Clock_Init              ; 第 3 步：时钟（必须最先）
218
219                  BL      GPIO_Init               ; 第 4 步：RCC + GPIO 复用（PC7 / PA2）
220                  BL      DMA2_Stream1_Init       ;         DMA（要在 USART6 开 DMAR 之前配好）
221                  BL      USART6_Init             ;         接收：100k 8E2 + IDLE 中断
222                  BL      NVIC_Init               ;         NVIC（最后开，避免半初始化就进中断）
223
224  Main_Loop
225                  BL      SBUS_Task               ; 飞控前处理：有数据就解析，没数据立即返回
226
227  ;---------------- 这里放飞控的其他任务（姿态解算、PID、电机输出…）------
228  ;    例：    BL    IMU_Update
229  ;            BL    PID_Control
230  ;            BL    Motor_Output
231  ;-----------------------------------------------------------------------
232
233                  B       Main_Loop
234
```

MAIN.S —— 时钟配置

- “复位时，16 MHz 的内部 RC 振荡器被选为默认的 CPU 时钟。”
- “随后，应用程序可以选择将 RC 振荡器或外部 4-26 MHz 时钟源作为系统时钟。”

3.11 Clocks and startup

On reset the 16 MHz internal RC oscillator is selected as the default CPU clock. The 16 MHz internal RC oscillator is factory-trimmed to offer 1% accuracy at 25 °C. The application can then select as system clock either the RC oscillator or an external 4-26 MHz clock source. This clock can be monitored for failure. If a failure is detected, the system automatically switches back to the internal RC oscillator and a software interrupt is generated (if enabled). This clock source is input to a PLL thus allowing to increase the frequency up to 84 MHz. Similarly, full interrupt management of the PLL clock entry is available when necessary (for example if an indirectly used external oscillator fails).

Several prescalers allow the configuration of the two AHB buses, the high-speed APB (APB2) and the low-speed APB (APB1) domains. The maximum frequency of the two AHB buses is 84 MHz while the maximum frequency of the high-speed APB domains is 84 MHz. The maximum allowed frequency of the low-speed APB domain is 42 MHz.

The devices embed a dedicated PLL (PLLI2S) which allows to achieve audio class performance. In this case, the I^2S master clock can generate all standard sampling frequencies from 8 kHz to 192 kHz.

数据拼接

```
379
380 // frame 指向 25 字节帧的起始地址（frame[0] 是帧头 0x0F）
381 // channels 是存放 16 个通道结果的数组
382 void SBUS_Unpack(uint8_t *frame, uint32_t *channels) {
383     int bitpos = 0;              // 对应 R1: 全局位指针
384     uint32_t *ptr = channels;    // 对应 R2: 输出指针
385     int count = 16;              // 对应 R3: 还要解析 16 个通道
386
387     while (count > 0) {
388         // 1. 计算当前通道在哪个字节，以及在这个字节里的偏移
389         int byte_index = bitpos / 8;     // 对应 R4: MOV R4, R1, LSR #3
390         int bit_offset = bitpos % 8;     // 对应 R5: AND R5, R1, #7
391
392         // 2. 一次性捞出 3 个字节，防止 11 bit 跨越 3 个字节的边界
393         // 注意: frame[0] 是帧头，所以数据从 frame[1] 开始
394         uint32_t b1 = frame[1 + byte_index]; // 对应 R7
395         uint32_t b2 = frame[2 + byte_index]; // 对应 R8
396         uint32_t b3 = frame[3 + byte_index]; // 对应 R9
397
398         // 3. 拼成 24 位小端整数 (对应 ORR 指令)
399         uint32_t temp = b1 | (b2 << 8) | (b3 << 16);
400
401         // 4. 右移对齐 (对应 MOV R7, R7, LSR R5)
402         temp = temp >> bit_offset;
403
404         // 5. 截取低 11 位
405         // 汇编中用 LSL #21 / LSR #21 实现，C 语言直接按位与更直观
406         uint32_t channel_value = temp & 0x07FF;
407
408         // 6. 写入输出数组，指针后移
409         *ptr = channel_value;            // 对应 STR R7, [R2], #4
410         ptr++;
411
412         // 7. 更新全局位指针，准备提取下一个通道
413         bitpos += 11;                    // 对应 ADD R1, R1, #11
414         count--;                         // 对应 SUBS R3, R3, #1
415     }
416 }
```

数据拼接

```
775  SBUS_Unpack
776                  MOV     R1, #0                  ; bitpos = 0
777                  LDR     R2, =channels           ; 输出指针
778                  MOV     R3, #16                 ; 还有 16 个通道
779  SU_Loop
780                  MOV     R4, R1, LSR #3          ; byte_index
781                  AND     R5, R1, #7              ; bit_offset
782                  ADD     R6, R0, R4              ; &frame[byte_index]
783                  LDRB    R7, [R6, #1]            ; frame[1+byte_index]
784                  LDRB    R8, [R6, #2]            ; 下一个字节
785                  LDRB    R9, [R6, #3]            ; 再下一个
786                  ORR     R7, R7, R8, LSL #8      ; 拼成 24 位小端值
787                  ORR     R7, R7, R9, LSL #16
788                  MOV     R7, R7, LSR R5          ; 右移 bit_offset 位
789                  MOV     R7, R7, LSL #21         ; 取低 11 位:
790                  MOV     R7, R7, LSR #21         ;   左移 21 再右移 21
791                  STR     R7, [R2], #4            ; channels[n] = value
792                  ADD     R1, R1, #11             ; 下一个通道起始位
793                  SUBS    R3, R3, #1
794                  BNE     SU_Loop
795
```